

Shenzhen Dudumiao Technology Co., Ltd.

smokefire Fire, Smoke & Smoking Detection

Deployment Guide

Document version: V1.0

Applicable software version: 1.0.0

Updated: 2026-08-03

Contents

Document information	3
1. Delivery and requirements	3
1.1 Release assets	3
1.2 Host requirements	4
2. Download and verification	4
2.1 CPU bundle	4
2.2 GPU bundle	5
2.3 Verify files	5
3. CPU/GPU installation (Docker)	6
3.1 Installation overview	6
3.2 Windows preparation	7
3.3 Windows installation	7
3.4 Linux installation	8
3.5 The five start-script stages	8
3.6 Lifecycle and logs	9
4. Source deployment (no Docker)	10
4.1 Prerequisites	10
4.2 Download and extract	10
4.3 One command	11
4.4 Where data and configuration live	11
4.5 Starting automatically	12
4.6 The one functional difference from the Docker route	12
4.7 When something fails	12
5. Video integration modes	13
6. Existing go2rtc integration	13
6.1 How the upstream project integrates	13
6.2 Configure once	13

6.3 Validate automatic import 14

7. Bundled go2rtc and direct RTSP 14

7.1 Bundled go2rtc 14

7.2 Direct RTSP 15

8. First start and acceptance 15

9. Configuration and LAN access 15

10. Routine operation 16

11. Backup, restore, and upgrade 16

12. Troubleshooting 16

13. Handover checklist 17

Installation 17

Video and business acceptance 17

Important

smokefire is a video-AI assistance and alerting tool. It is not a certified fire alarm device and does not replace statutory fire protection, inspections, emergency procedures, or human verification. False positives and missed detections remain possible.

Document information

Item	Value
Audience	Deployment engineers, administrators, and site operators
Delivery	Either an offline Docker bundle (CPU / NVIDIA GPU) or a source bundle that needs no Docker
Hosts	Windows 10/11, Windows Server, mainstream x86_64 Linux
Video input	Direct RTSP, bundled go2rtc, or bulk sync from an existing go2rtc
Default URL	http://127.0.0.1:8600
Project	https://github.com/newtv-ai/smokefire

1. Delivery and requirements

1.1 Release assets

GitHub Release [v1.0.0](#) offers two deployment routes. Pick the one that fits the target host.

A. Offline Docker bundle (no Python on the host, no dependency download):

- [smokefire-deploy-1.0.0-cpu.zip](#): complete CPU bundle;
- [smokefire-deploy-1.0.0-gpu.zip](#): GPU configuration, models, scripts, and manuals;
- [smokefire-images.tar.gz.partNN](#): all GPU image parts, required with the GPU bundle;
- [SHA256SUMS-v1.0.0.txt](#): checksums for Release assets.

Each bundle includes prebuilt smokefire and go2rtc Docker images, both model weights, Compose files, Windows/Linux scripts, and all six Chinese/English PDFs. Nothing is fetched from GitHub after installation. See section 3.

B. Source bundle (for hosts that should not run Docker):

- [smokefire-source-1.0.0.zip](#): about 90 MB, containing the application source, front end, both model weights, install and start scripts, and the manuals.

The source bundle is far smaller, but the host must provide Python and ffmpeg, and the **first dependency install needs internet access** (about 1 GB for CPU, 3 GB for GPU). See section 4.

Both routes deliver the same features; only the dependency isolation differs. The single functional difference is described in 4.6.

1.2 Host requirements

Item	CPU bundle	GPU bundle
CPU	x86_64, 4+ cores recommended	x86_64, 4+ cores recommended
RAM	8 GB minimum, 16 GB recommended	16 GB minimum
GPU	Not required	NVIDIA GPU, 8 GB+ VRAM recommended
Driver	-	Host driver must support the bundled CUDA 12.8 runtime
Docker	Engine/Desktop 24+ and Compose v2	Same, plus NVIDIA Container Toolkit
Free disk	20 GB plus event data	50 GB plus event data
Cameras	RTSP; H.264 recommended	Same

The CPU build suits pilots and smaller sites. GPU compatibility and capacity are not promised by reference to any single validation-host model. First confirm that the target driver can run the bundled CUDA 12.8 runtime, then validate startup, both model loads, VRAM use, sustained operation, and the planned stream count with real feeds. Actual capacity depends on the GPU, driver, codec, resolution, sampling interval, recording, and network conditions.

2. Download and verification

The repository is currently Public. Users can open the repository and download Release [v1.0.0](#) without signing in to GitHub:

```
https://github.com/newtv-ai/smokefire
```

Release assets, download the row that matches your build:

Asset	Size	CPU	GPU
smokefire-deploy-1.0.0-cpu.zip	~913 MB	required	-
smokefire-deploy-1.0.0-gpu.zip	~90 MB	-	required
smokefire-images.tar.gz.part01	1.80 GB	-	required
smokefire-images.tar.gz.part02	1.80 GB	-	required
smokefire-images.tar.gz.part03	1.16 GB	-	required
SHA256SUMS-v1.0.0.txt	~1 KB	recommended	recommended

The CPU bundle ships its image inside the zip. The GPU image exceeds the GitHub per-file limit, so it is published as three separate parts: the GPU build needs one zip plus all three parts, and a missing part makes the import impossible.

Install into a path without spaces or non-ASCII characters, for example [D:\smokefire](#) on Windows or [/opt/smokefire](#) on Linux.

2.1 CPU bundle

- 1 Open the Releases page and expand the Assets list of `v1.0.0`.
- 2 Download `smokefire-deploy-1.0.0-cpu.zip` and `SHA256SUMS-v1.0.0.txt`.
- 3 Extract the zip into the target directory.

Extracted layout:

```
D:\smokefire\  
docs\          manual PDFs, three per language  
images\        offline image, included in the CPU bundle  
models\        both model weights  
.env.example    configuration template  
docker-compose.yml  
docker-compose.gpu.yml  
start.ps1      start.sh  
stop.ps1       stop.sh  
verify.ps1     verify.sh  
configure-go2rtc.ps1  configure-go2rtc.sh  
SHA256SUMS
```

2.2 GPU bundle

- 1 Download `smokefire-deploy-1.0.0-gpu.zip`. It contains configuration, models, scripts, and manuals, but no image.
- 2 Download all three parts `smokefire-images.tar.gz.part01`, `part02`, and `part03`, about 4.8 GB in total.
- 3 Download `SHA256SUMS-v1.0.0.txt`.
- 4 Extract the GPU bundle into the target directory.
- 5 Move all three parts into the extracted `images/` directory.

Keep the original filenames. Do not rename, extract, or merge the parts yourself. The directory must look like this:

```
D:\smokefire\images\  
README.txt  
smokefire-images.tar.gz.part01  
smokefire-images.tar.gz.part02  
smokefire-images.tar.gz.part03
```

The start script assembles the parts in filename order, loads the Docker images, and removes only its own temporary assembled file. The original parts remain intact. Assembly needs about 4.8 GB of extra free space inside `images/`.

2.3 Verify files

There is no separate verification step. The start script verifies the delivery files as its first stage and stops with the offending filename if anything fails, so a normal deployment is a single command; see section 3.

The automatic verification prints one summary line:

```
Verified 25 files.
```

The CPU bundle verifies 22 files and the GPU bundle 25, the difference being the three image parts. A lower count means a file is not in place.

Run the verification by hand only when you want to confirm a download without starting the service yet:

```
powershell -ExecutionPolicy Bypass -File .\verify.ps1
```

```
chmod +x verify.sh start.sh stop.sh configure-go2rtc.sh
./verify.sh
```

Fixed model checksums:

File	Size	SHA-256
models/fire_smoke_v5.pt	44,014,233 bytes	b5248d55c5d90e341bc4e537887d4bce4b9af1bd0330ddd87825294750a1e216
models/smoking_v4.pt	44,025,689 bytes	0ea7b66c2105f1898f56f828b18ffb01934fb1f47a795192e8867ae8fcb128dd

Do not continue after any checksum failure. Download the affected file again. An interrupted part download is the most common cause; re-download only the failing part, not the whole bundle. Continue to section 3 once verification passes.

3. CPU/GPU installation (Docker)

3.1 Installation overview

CPU or GPU, Windows or Linux, the main line is the same: install Docker, then run the start script once. Verification, image import, container startup and the readiness wait all happen inside that one script, which runs five internal stages and prints [N/5] for each one.

Step	Action	Command	Expected time
1	Install and start Docker	See 3.2 / 3.4	10-30 min including download
2	Run the start script (it verifies first)	start.ps1 / start.sh	~5 min CPU, ~15 min GPU
3	Open the web page	Browse to http://127.0.0.1:8600	1 min
4	Choose a video mode	configure-go2rtc.*, see section 5	5 min and up

Most of the first-run time is spent importing the offline Docker images, which is expected. Image footprint after import:

Image	Size on disk
smokefire:1.0.0-cpu	~3.1 GB
smokefire:1.0.0-gpu	~13.3 GB
alexxit/go2rtc:1.9.9	~0.4 GB

Peak disk use during the first GPU start is about 4.8 GB of parts plus 4.8 GB of temporary assembled archive plus 13.7 GB of images. Keep at least 25 GB free on the Docker data disk, excluding event recordings.

3.2 Windows preparation

- 1 Install Docker Desktop and keep the default WSL 2 backend.
- 2 Start Docker Desktop and wait until the status in the lower-left corner reads Running.
- 3 Confirm Linux container mode. If the tray context menu offers "Switch to Windows containers...", you are already in Linux mode, so do not click it.
- 4 Verify in PowerShell:

```
docker version
docker compose version
```

`docker version` must print both a Client and a Server section. A Client-only output means the engine is not up; go back to step 2 and wait for Docker Desktop to report Running.

The GPU build needs three more steps:

- 1 Install or update the NVIDIA driver. It must support the CUDA 12.8 runtime inside the delivered image.
- 2 In Docker Desktop, Settings, Resources, confirm that GPU support is enabled. The WSL 2 backend enables it by default.
- 3 Confirm the host sees the GPU:

```
nvidia-smi
```

`nvidia-smi` must list the GPU model and driver version. Do not continue while this fails; fix the driver first. The in-container CUDA check needs the image to be imported already, so it comes after the start script; see 3.5.

3.3 Windows installation

Run the commands below **from an already open PowerShell window**. Do not double-click the script and do not use the right-click "Run with PowerShell" entry: in that mode the window closes the moment the script reports an error, the message is gone before it can be read, and the symptom looks like "it quit halfway through".

```
cd D:\smokefire
powershell -ExecutionPolicy Bypass -File .\start.ps1
```

That is the whole command. Verifying the delivery files is the start script's first stage, so `verify.ps1` does not need a separate run.

`-ExecutionPolicy Bypass` applies to that single invocation and does not change the system execution policy.

The script records everything to `start-log.txt` in the deployment directory. If anything goes wrong, sending that single file is enough for support: no screenshots, no recalling what scrolled past.

The script ends on a "Press Enter to close this window" prompt, so the window survives even when the script is double-clicked. Pass `-NoPause` to skip that prompt in automated use.

Every failing step prints its cause and stops, so the script never continues with a half-built environment. If it stops, read the last `[N/5]` line in the terminal and match it against 3.5.

3.4 Linux installation

Install Docker Engine and Compose v2. For the GPU build, also install the NVIDIA driver and NVIDIA Container Toolkit. Verify:

```
docker version
docker compose version
nvidia-smi          # GPU bundle only
```

Install:

```
cd /opt/smokefire
chmod +x verify.sh start.sh stop.sh configure-go2rtc.sh
./start.sh
```

Keep `verify.sh` in the `chmod` list: the start script calls it, even though you never run it yourself.

Prefix the docker commands with `sudo` when the current user is not in the `docker` group. The stage output matches 3.5.

3.5 The five start-script stages

`start.ps1` and `start.sh` behave identically and print `[1/5]` through `[5/5]`. Below is what each stage does, its normal output, and what to do when it stops.

[1/5] Verify delivery files

```
[1/5] Verifying deployment files...
Verified 25 files.
```

The script verifies for you, so nothing has to be run beforehand. The CPU bundle reports 22 files and the GPU bundle 25. Stopping here means a file is missing or its checksum does not match; the script names the file, and 2.3 explains how to re-download it.

[2/5] Check Docker

```
[2/5] Checking Docker...
```

Checks `docker info` and `docker compose version`. Stopping here means the Docker engine is not running or Compose v2 is missing, and the script states which one.

The first run also prints one extra line here:

```
Created .env from .env.example
```

This means the deployment `.env` was generated from the template. The GPU template already contains `SMOKEFIRE_IMAGE=smokefire:1.0.0-gpu` and `COMPOSE_FILE=docker-compose.yml|docker-compose.gpu.yml`, so no manual edit is needed.

[3/5] Import the offline images

```
[3/5] Importing the prebuilt smokefire and go2rtc images when needed...
Assembling smokefire-images.tar.gz.part01...
Assembling smokefire-images.tar.gz.part02...
Assembling smokefire-images.tar.gz.part03...
Loaded image: smokefire:1.0.0-gpu
Loaded image: alexxit/go2rtc:1.9.9
```

The longest stage. The GPU build assembles the parts before importing, which can take well over ten minutes on a spinning disk. There is no progress bar during assembly; that is expected, so do not interrupt it. When both images already exist, this stage prints a single `Images ... are already available.` line and skips.

[4/5] Start the containers

```
[4/5] Starting smokefire...
Container smokefire-go2rtc-1 Started
Container smokefire-smokefire-1 Started
```

go2rtc starts first and smokefire waits for its health check, an ordering enforced by the compose dependency.

[5/5] Wait for readiness

```
[5/5] Waiting for readiness (up to 5 minutes)...
smokefire 1.0.0 is ready: http://127.0.0.1:8600
```

The script polls `/api/health/ready` every 5 seconds for up to 5 minutes. The first start loads both models, so one or two minutes is normal. On timeout the script prints the container status and the last 200 log lines before exiting; send that output to support for diagnosis.

For the GPU build, add one in-container check after the start script finishes:

```
docker run --rm --gpus all smokefire:1.0.0-gpu python -c "import torch;
print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

It must print `True` and the GPU model. `False` means the container did not get the GPU: return to 3.2 and check the driver and Docker Desktop GPU support. Never hand over a GPU deployment that is silently running on the CPU.

3.6 Lifecycle and logs

```
docker compose ps
docker compose logs -f smokefire
```

Use `stop.ps1` on Windows or `./stop.sh` on Linux. Stopping preserves business data. Never run `docker compose down -v`, which removes named volumes.

4. Source deployment (no Docker)

Use this route when the target host should not run Docker. The features are identical to the Docker route; only the dependency isolation differs. Docker seals Python and the system libraries inside an image, while this route uses the host's own Python.

	Docker route	Source route
Target host needs	Docker + Compose v2	Python, ffmpeg, plus libgl1 on Linux
Internet needed on first run	No, images ship with the bundle	Yes , about 1 GB of dependencies (3 GB for GPU)
Bundle size	~913 MB CPU / ~90 MB GPU plus 4.8 GB of parts	~90 MB
Upgrading	Swap the image	Extract a new directory and reinstall

4.1 Prerequisites

The install script checks each item and stops with the command to fix it.

Dependency	Why	How to install
Python	3.11 through 3.14 all work , each with its own hashed lock file; 3.12 or 3.13 is recommended as both are verified end to end. The script does not gate on the version. Without a matching lock file it installs from <code>requirements.txt</code> instead, with no pinning and no hash verification, and says so	Windows <code>winget install Python.Python.3.12</code> ; Linux use the distribution package manager
ffmpeg + ffprobe	Required for the H.264 evidence chain of event recordings; both are needed	No need to install these first. If they are absent, the installer puts them inside <code>.venv</code> — no admin rights, no change to the system PATH. To use your own build instead, install it and put it on PATH first: Windows <code>winget install Gyan.FFmpeg</code> ; Debian/Ubuntu <code>sudo apt-get install -y ffmpeg</code>
libgl1, libglib2.0-0	Required by OpenCV, Linux only	<code>sudo apt-get install -y libgl1 libglib2.0-0</code>
Internet access	First dependency install only	~1 GB CPU, ~3 GB GPU
NVIDIA driver	GPU only , must support CUDA 12.8	Vendor driver

4.2 Download and extract

Download `smokefire-source-1.0.0.zip` (~90 MB) from the same Release and extract it into a path without spaces or non-ASCII characters.

```
D:\smokefire-source smokefire\      application source
web\                                front-end pages
models\                             both model weights
deploy\                             install and service-registration scripts
docs\                               Chinese and English manual PDFs
requirements-*.lock                 lock files for Python 3.12
requirements\                       lock files for 3.10, 3.11, 3.13 and 3.14
.env.example                        configuration template
start.ps1 start.sh                  one-command entry points
SHA256SUMS
```

4.3 One command

Windows, from an already open PowerShell window:

```
cd D:\smokefire-source
powershell -ExecutionPolicy Bypass -File .\start.ps1      # CPU
powershell -ExecutionPolicy Bypass -File .\start.ps1 -Gpu # NVIDIA GPU
```

Linux:

```
cd /opt/smokefire-source
chmod +x start.sh
./start.sh      # CPU
./start.sh --gpu # NVIDIA GPU
```

On a slow link, add the mirror flag; it routes both PyPI and the PyTorch wheels through a mainland-China mirror:

```
powershell -ExecutionPolicy Bypass -File .\start.ps1 -CnMirror
```

```
./start.sh --cn-mirror
```

The largest dependency is torch (about 110 MB for the CPU wheel alone, more for GPU). It is not hosted on PyPI, so `-PipMirror` does not cover it — that is why the combined flag exists. To point at different sources, override them separately with `-PipMirror` / `--pip-mirror` (PyPI) and `-TorchMirror` / `--torch-mirror` (the torch wheel directory).

The **first run** creates `.venv`, installs the locked dependencies (measured at about 1.1 GB for CPU, a good ten minutes depending on the link) and then starts the service. **Every later run** of the same command skips the install and starts straight away, with no internet needed.

This line means it is up; open that address in a browser:

```
"msg": "smokefire ready", "port": 8600
```

Press Ctrl+C to stop. Delete the `.venv` directory to force a clean reinstall.

4.4 Where data and configuration live

Item	Location
Database and event recordings	<code>data/</code> inside the extracted directory
Model weights	<code>models/</code> inside the extracted directory
Configuration	<code>.env</code> , generated from <code>.env.example</code> on first start
Startup log	<code>start-log.txt</code> in the extracted directory (Windows only)

Everything stays inside the extracted directory; nothing is written to system locations. To move the installation, copy the whole directory but leave `.env` behind and rerun the one command on the new host so it is rebuilt there.

To change the port or allow LAN access, edit `SMOKEFIRE_HOST`, `SMOKEFIRE_PORT` and `SMOKEFIRE_ALLOWED_HOSTS` in `.env` and restart. Their meaning is covered in section 9.

4.5 Starting automatically

A foreground service stops when the terminal closes. For unattended operation, register it:

- Windows: ``deploy\windows`

`egister-service.ps1`, which needs administrator rights and NSSM (winget install NSSM)

- Linux: adjust the paths in `deploy/linux/smokefire.service` as its comments describe, then `sudo systemctl enable --now smokefire`

4.6 The one functional difference from the Docker route

Bundled go2rtc mode requires you to supply a go2rtc executable yourself and run it at the address `SMOKEFIRE_GO2RTC_API` points to in `.env`. The Docker route ships a go2rtc container instead. The other two video modes, direct RTSP and an existing site go2rtc, behave exactly as sections 6 and 7 describe.

4.7 When something fails

`start.ps1` records the whole run to `start-log.txt`; send that one file to support. On Linux use:

```
./start.sh 2>&1 | tee start-log.txt
```

Symptom	Cause	Action
Message that no lock file matches the interpreter	This Python version has no pregenerated lock file	Not an error: the install continues without hash verification. Switch to 3.12 or 3.13 for the fully verified path
Bundled ffmpeg reported unusable	No prebuilt binary for this architecture (for example arm64)	Install ffmpeg yourself per 4.1, then open a new terminal and rerun
THESE PACKAGES DO NOT MATCH THE HASHES	The download was cut short, so hash verification rejected the truncated file	Just rerun. Packages already downloaded sit in the pip cache and are not fetched again — only the incomplete one is. If it keeps failing, add <code>-CnMirror</code> / <code>--cn-mirror</code>
Dependency install fails partway	Usually the network	Retry with <code>-CnMirror</code> / <code>--cn-mirror</code>
Linux reports a missing libGL	System library not installed	<code>sudo apt-get install -y libgl1 libglib2.0-0</code>

5. Video integration modes

Choose one mode for the site:

Mode	Best fit	Configuration effort	Data path
A. Direct RTSP	A few cameras and no go2rtc	One URL per camera	Camera/NVR -> smokefire
B. Bundled go2rtc	Cameras are managed in smokefire; detection and recording should share one producer	One URL per camera in smokefire	Camera -> bundled go2rtc -> detection/recording
C. Existing go2rtc	The customer already manages feeds in go2rtc	One API base plus one RTSP base	Existing go2rtc -> automatic import -> smokefire

For an existing go2rtc site, use mode C. Do not copy every camera RTSP URL into smokefire.

The provided configuration script changes only `.env` in the deployment directory. If the service is already running, it recreates the smokefire container while preserving business data.

6. Existing go2rtc integration

6.1 How the upstream project integrates

smokefire retains the upstream system's bulk synchronization approach:

```
Customer go2rtc GET /api/streams
    |
    v
smokefire synchronizes main-stream names and states
    |
    v
rtsp://customer-go2rtc:8554/<URL-encoded-stream-name>
    |
    v
AI detection, recording, preview, and events
```

The synchronizer only reads `GET /api/streams`; it never calls create, update, or delete operations on the customer's go2rtc. The first sync registers all main streams automatically. When a matching main stream exists, names ending in `_sub`, `-sub`, or `_sd` are treated as substreams and skipped to prevent duplicates. At the default 300-second interval, new streams are added, missing streams are disabled, and returning streams are enabled again.

6.2 Configure once

If go2rtc runs on the same Docker host, do not use `127.0.0.1` from inside the smokefire container. Use `host.docker.internal`:

```
.\configure-go2rtc.ps1 -Mode upstream `
  -Api http://host.docker.internal:1984 `
  -Rtsp rtsp://host.docker.internal:8554
```

Linux:

```
./configure-go2rtc.sh upstream \
  http://host.docker.internal:1984 \
  rtsp://host.docker.internal:8554
```

For go2rtc on another LAN host, use its LAN address, for example:

```
API http://192.168.10.20:1984
RTSP rtsp://192.168.10.20:8554
```

Equivalent `.env` entries:

```
SMOKEFIRE_GO2RTC_ENABLED=false
SMOKEFIRE_UPSTREAM_GO2RTC_ENABLED=true
SMOKEFIRE_UPSTREAM_GO2RTC_API=http://192.168.10.20:1984
SMOKEFIRE_UPSTREAM_GO2RTC_RTSP=rtsp://192.168.10.20:8554
SMOKEFIRE_UPSTREAM_GO2RTC_SYNC_INTERVAL_SEC=300
```

6.3 Validate automatic import

- 1 Open <http://<go2rtc-host>:1984/api/streams> and confirm it returns the expected feeds.
- 2 Run the configuration script and start smokefire.
- 3 Open Camera Management and confirm main streams appear automatically with the Upstream badge.
- 4 Confirm matching substreams were not duplicated.
- 5 Validate at least one preview, AI inference result, and event recording.
- 6 Add a test stream and confirm the next sync imports it; remove and restore it to validate automatic disable/re-enable.

Keep existing go2rtc stream names unique and stable. The synchronizer owns imported source URLs; do not create manual cameras with the same names.

7. Bundled go2rtc and direct RTSP

7.1 Bundled go2rtc

Use this when the site has no go2rtc but wants detection and recording to share one upstream camera producer:

```
.\configure-go2rtc.ps1 -Mode builtin
```

```
./configure-go2rtc.sh builtin
```

Continue adding camera RTSP URLs in Camera Management. smokefire creates internal `sf-camN` aliases automatically, and both AI and recording consume the bundled go2rtc output.

7.2 Direct RTSP

```
.\configure-go2rtc.ps1 -Mode direct
```

```
./configure-go2rtc.sh direct
```

Add camera or NVR RTSP URLs individually in Camera Management. This is the simplest mode, but AI and recording may create separate upstream sessions. Prefer go2rtc as the stream count grows.

8. First start and acceptance

Open <http://127.0.0.1:8600> on the service host. Initial model loading may take several minutes.

Endpoint	Purpose	Expected
/api/health/live	Process liveness	HTTP 200
/api/health/ready	Models, scheduler, recording, disk, and upstream sync	HTTP 200
/api/status	Runtime and notification metrics	JSON response

Minimum acceptance:

- 1 Live Wall, Event Center, and Camera Management open successfully.
- 2 Video is connected through the selected mode; an existing-go2rtc deployment proves bulk automatic import.
- 3 Cameras become Online with correct names, images, and timestamps.
- 4 Authorized test material exercises both the fire/smoke and smoking models.
- 5 Event details contain snapshot, class, confidence, time, and recording.
- 6 Confirm True, Mark False Positive, reconnect behavior, restart, and data persistence work.
- 7 Run the planned stream count and record throughput, latency, GPU/CPU, disk, and network behavior.

Installation success is not production acceptance. See the Model Capability Test Guide for model boundaries and site test design.

9. Configuration and LAN access

The Web port binds to `127.0.0.1:8600` by default. For LAN access, an engineer must set the bind address and allowed hostnames in `.env`, then restart.

Setting	Default	Purpose
SMOKEFIRE_IMAGE	Set by CPU/GPU bundle	Application image
SMOKEFIRE_ALLOWED_HOSTS	localhost,127.0.0.1	Accepted Web Host values
SMOKEFIRE_GO2RTC_ENABLED	false	Enables bundled go2rtc fan-out
SMOKEFIRE_UPSTREAM_GO2RTC_ENABLED	false	Enables existing-go2rtc synchronization
SMOKEFIRE_UPSTREAM_GO2RTC_API	Empty	Existing go2rtc API base
SMOKEFIRE_UPSTREAM_GO2RTC_RTSP	Empty	Existing go2rtc RTSP base
SMOKEFIRE_INFER_WORKERS	4	Inference workers
SMOKEFIRE_INFER_BATCH_MAX	8	Maximum inference batch
SMOKEFIRE_STOP_GRACE_PERIOD	180s	Graceful shutdown period

RTSP URLs may contain credentials. Restrict access to the deployment directory, backups, and terminal history, and redact full credentials from screenshots and support tickets.

10. Routine operation

```
docker compose ps
docker compose logs --tail 200 smokefire
curl http://127.0.0.1:8600/api/health/ready
```

Daily checks should cover camera status, latest successful upstream sync, preview freshness, event playback, notification queues, and disk high-water state. In existing-go2rtc mode, compare the number of go2rtc main streams with Upstream cameras in smokefire.

11. Backup, restore, and upgrade

Business data is stored in the `smokefire-data` named volume. Stop the service, record the version, and copy the entire `/app/data` tree. A complete backup includes camera records, the SQLite database, event videos, snapshots, false-positive feedback, and notification outbox. The customer's existing go2rtc configuration remains the customer's backup responsibility and is not stored in the smokefire volume.

Restore data to the matching software version, then validate health, cameras, historical events, recordings, feedback, and upstream sync.

Before upgrading, verify that the backup is readable. Download and verify the new bundle, stop the old version, start from a new directory, and run minimum acceptance. Roll back to the prior image and backup if validation fails.

12. Troubleshooting

Symptom	First check	Action
Docker command unavailable	Docker Desktop/Engine	Start the engine and retry docker version
Start script exits before finishing	start-log.txt in the deployment directory	It records the whole run; send that file to support
Unclear which step failed	The last [N/5] line, or start-log.txt	Match it against 3.5 stage by stage
GPU container unavailable	Driver, Container Toolkit, Docker GPU support	Pass nvidia-smi and container CUDA checks first
GPU image part missing	images/ , filenames, SHA-256	Download all parts without renaming them
Readiness remains false	Logs, models, disk, upstream sync	Run docker compose logs --tail 300 smokefire
Existing go2rtc imports nothing	API URL, container reachability to 1984, /api/streams	Use host.docker.internal on the same host or the LAN IP remotely
Fewer imports than go2rtc names	Main/substream naming	_sub , -sub , and _sd matches are deliberately deduplicated
Upstream camera becomes disabled	Feed disappeared or sync failed	Restore the feed; the next successful sync re-enables it
Camera keeps reconnecting	RTSP 8554, encoded stream name, source state	Test the same go2rtc RTSP URL from the deployment host
Too many or no alerts	Threshold, target pixels, scene interference	Replay representative positive and negative samples; do not lower thresholds blindly
Event has no video	Disk, warm-up, source stability	Inspect disk status and FFmpeg logs

13. Handover checklist

Installation

- ☐ The complete CPU or GPU offline delivery was selected and verified.
- ☐ smokefire, go2rtc, and both model checks passed.
- ☐ Start, stop, restart, and host reboot recovery were tested.
- ☐ [/api/health/live](#) and [/api/health/ready](#) return normally.
- ☐ Install directory, data volume, backup location, and version 1.0.0 are recorded.
- ☐ The repository remains Public, and anonymous users can open the repository, Release page, and checksum download.

Video and business acceptance

- ☐ Direct RTSP, bundled go2rtc, or existing go2rtc mode is explicitly selected.
- ☐ Existing-go2rtc sites validated one-time setup, bulk main-stream import, addition, removal, and recovery.
- ☐ Planned cameras have correct names, time, image, inference, and recording.

- ☐ Both models were tested with representative positive and negative samples.
 - ☐ Snapshots, recordings, Confirm True, false-positive archive, and feedback export work.
 - ☐ Planned capacity, reconnect behavior, and continuous operation were tested.
 - ☐ Users received the User Operation Manual and Model Capability Test Guide and completed handover training.
-

Prepared by: Shenzhen Dudumiao Technology Co., Ltd. Project:
<https://github.com/newtv-ai/smokefire>